

Assignment 3:

Understanding Algorithm Efficiency and Scalability

Shreeya Gadgil

For MSCS 532 – Algorithms and Data Structures

A) Quick Sort Analysis:

a. Average case Time complexity Analysis:

- i. Methodology: Pick one element from the array as a reference point or pivot. Then rearrange the array such that all values smaller than the pivot move to its left and values larger than the pivot move to the right. This locks the pivot in its sorted position. Then apply the same steps separately to the left sub-array and right sub-array. Stop the process when a sub-array has a single element, since a single item is already considered sorted.

ii. Mathematical analysis:

- Let $Z = [z_1, z_2, z_3, \dots, z_n]$ denote the elements of an array A sorted in ascending order. Let $Z_{ij} = \{z_i, z_{i+1}, \dots, z_j\}$ be the set of elements between z_i and z_j both inclusive. We define an indicator random variable X_{ij} as:

$$X_{ij} = \begin{cases} 1, & \text{if } z_i \text{ is compared to } z_j \\ 0, & \text{otherwise} \end{cases}$$

The total number of comparisons X is the sum of all pairs:

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}$$

- **Linearity of Expectation:**

Taking the expected value of both sides

$$\mathbb{E}[X] = \mathbb{E}\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \mathbb{E}[X_{ij}]$$

Since $\mathbb{E}[X_{ij}] = 1 \cdot \mathbb{P}(\text{when } z_i \text{ is compared to } z_j) + 0 \cdot \mathbb{P}(\text{when } z_i \text{ is not compared to } z_j)$, we have

$$\mathbb{E}[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \mathbb{P}(\text{when } z_i \text{ is compared to } z_j)$$

- **Probability calculation:**

- Two elements z_i and z_j are compared only if the first element chosen as a pivot from the subset is either one of them.
 - If any element between z_i and z_j is chosen first, z_i and z_j will be separated into different sub-arrays and will never be compared.
 - As the total number of elements is $j-i+1$, when the pivot is chosen uniformly at random, every element in Z_{ij} has an equal probability of $1/(j-i+1)$ of being chosen first.
 - Hence the probability of either z_i or z_j being chosen as the first pivot is $2/(j-i+1)$
- Substituting the probability back into the expectation formula:

$$\mathbb{E}[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1}$$

Let $k = j - i$. As j goes from $i+1$ to n , k goes from 1 to $n-i$:

$$\mathbb{E}[X] = \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} < \sum_{i=1}^n \sum_{k=1}^n \frac{2}{k} = \sum_{i=1}^n 2 \cdot H_n$$

where $H_n = \sum_{k=1}^n \frac{1}{k}$ is the nth Harmonic number, bound by $\ln(n) + O(1)$

$$\mathbb{E}[X] < 2n \cdot H_n = 2n(\ln(n) + O(1)) = \Theta(n \log n)$$

This proves that the average case time complexity of Randomized Quicksort is $O(n \log n)$.

iii. Algorithmic comparison:

Both algorithms share the same partition logic and the same recursion structure. The only difference is the pivot rule. Both were benchmarked across four input sizes and four data set types of random, sorted, reverse sorted and repeated elements.

- **Random data:**

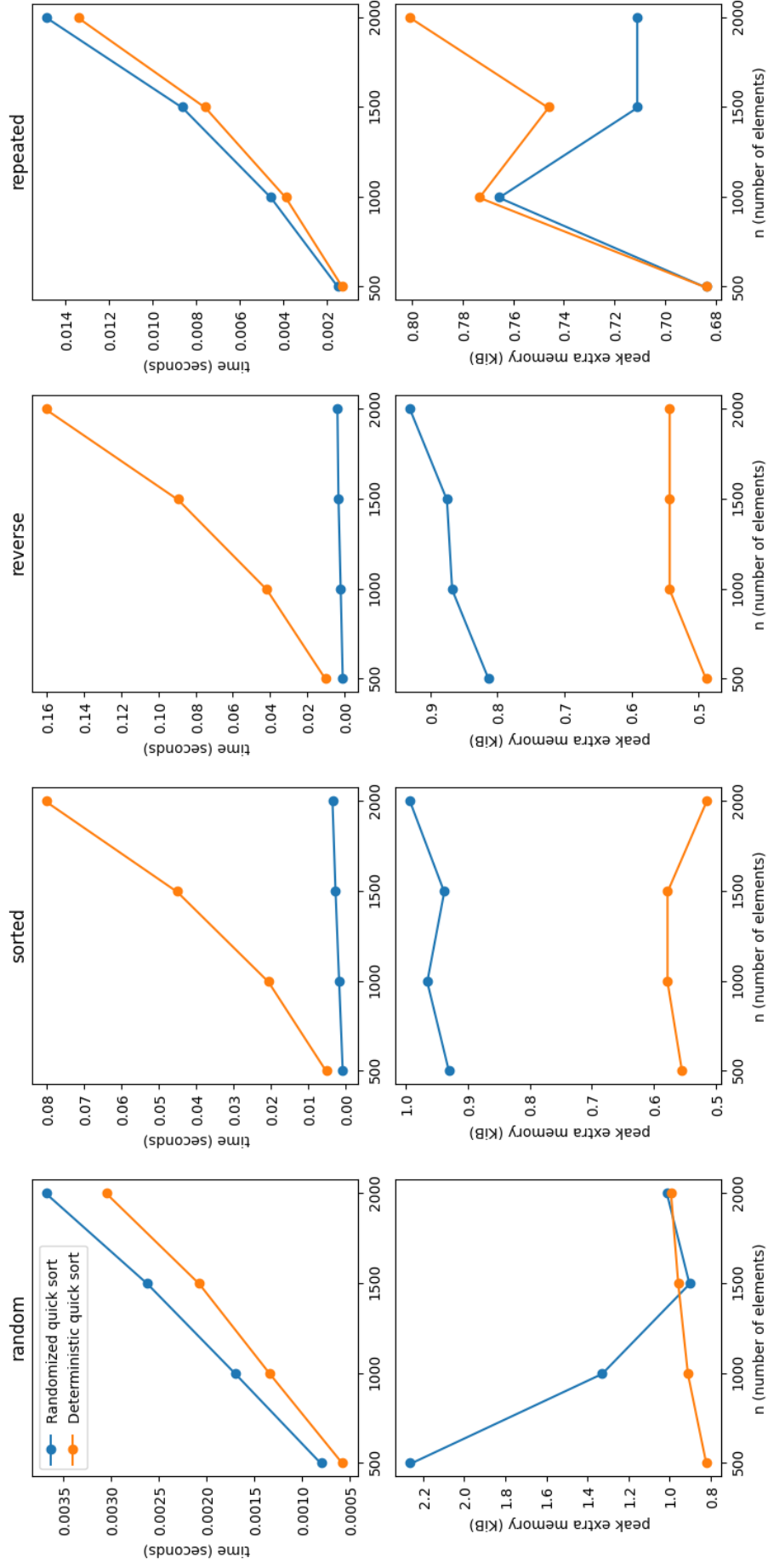
- Both algorithms grow with n as expected, but deterministic was consistently slightly faster than randomized.
- On random input the first element of a random subarray is itself effectively a random pivot, so deterministic quicksort gets the same average case behaviour whereas randomized needs the extra step of `random.randint` on every partition call.

- **Sorted and reverse sorted data:**

- This is where the two algorithms diverge. Deterministic quicksort degraded to its worst case, while randomized quicksort stayed low and flat on both.
- This proves that a fixed pivot rule can be vulnerable to specific inputs, a randomized rule is not.

- **Repeated elements:**

- Rather than one algorithm staying flat and the other exploding, both algorithms grew together.
- Tracing through the partition function on an array of all-equal elements shows that the test `a[j] <= x` is True for every element, regardless of which index was chosen as pivot. By the time the loop runs, the pivot value is already fixed, and the loops behaviour depends on the value, not the index. Hence, randomizing which index gets swapped to the end is irrelevant when every index holds the same value.
- This traces back to the assumption at the start of the average-case proof, it requires distinct elements. With duplicates, both algorithms degrade to $\Theta(n^2)$.



B) Hashing with Chaining

a. Expected times under Uniform Hashing:

- i. Under the assumption of Simple Uniform Hashing, any given key can hash into any of the m slots in a hash table, independent of where any other key has hashed.
- ii. The performance of hash table operations depends on the load factor(α), which is calculated as:

$$\alpha = n/m$$

n – total number of elements currently stored

m – total number of slots in the table

b. Collision resolution by chaining - Expected time complexity

In a chained hash table, each slot would contain a pointer to a linked list of elements that hash to that slot. The expected length of any single linked list is equal to the load factor.

- i. Insert: $O(1)$ expected
 - New elements are inserted at the head of the linked list for a slot. Takes constant time, and is independent of α .
- ii. Unsuccessful search: $O(1 + \alpha)$ expected time
 - The algorithm hashes the key and scans the entire linked list at that slot. The expected number of elements examined is equal to the average list length, α . Adding $O(1)$ for hash function evaluation leads to $O(1 + \alpha)$
- iii. Successful search: $O(1 + \alpha)$ expected time
 - The algorithm scans until it finds the target element. On average, it scans half as many. The constant differs, but it leads to $O(1 + \alpha)$ expected time as well.
- iv. Delete: $O(1 + \alpha)$ expected time
 - The algorithm searches for the element through α elements, and then deletes it.

c. How load factor affects performance

- i. If m is fixed and n grows, α grows without bound, so every operation slows down in proportion. In the extreme ($m = 1$), everything sits in one chain and each operation is $\Theta(n)$.
- ii. If α is very small, operations are fast but most slots are empty and memory is wasted. Keeping α near a constant such as 1 balances speed against memory.
- iii. Example: 4000 keys in 8 slots gives $\alpha = 500$, so each chain holds about 500 entries.

d. Keeping the load factor low

- i. Dynamic resizing. When $\alpha > 1$, allocate a table of size $2m$ and rehash every key. Every key must be rehashed because $_hash$ depends on m .
- ii. A single resize costs $\Theta(n)$, but resizes happen at sizes 1, 2, 4, 8, ..., n , and that geometric series sums to about $2n$. The total resizing work over n inserts is $\Theta(n)$, so the amortized cost per insert is $O(1)$. Adding a fixed amount each time would need $\Theta(n)$ resizes and cost much more in total.
- iii. Universal hashing does not lower α , but it keeps any fixed set of keys from piling into a few slots. Resizing controls the average chain length, and the hash function controls how evenly keys spread.